Rethinking LLM Fine-Tuning via Weight Space Reparameterization: Preserving Safety during Downstream Adaptation

Anonymous Author(s)
Affiliation
Address
email

Abstract

Fine-tuning large language models (LLMs) on downstream tasks often weakens their previously aligned safety behavior, creating an inherent trade-off between task adaptation and safety preservation. Existing approaches attempt to mitigate this issue by identifying safety-relevant neurons, layers, or update directions in the original parameter space. However, safety-related information is often distributed across many directions in this space, causing downstream updates to overwrite parameters that encode safety behavior. We propose **WSR-Tune**, a Weight Space Reparameterization-based fine-tuning framework that preserves safety during downstream adaptation. Our approach constructs a safety-conditioned basis and reparameterizes the weight matrices such that safety-relevant information becomes concentrated in a small subset of directions. We then identify safety-critical directions and freeze them in the reparameterized space, while updating the remaining complementary directions for downstream adaptation. By explicitly structuring the parameter space to disentangle safety and task information, our method reduces interference between objectives and enables simultaneous preservation of safety and improvement in downstream performance. Experimental results show that our method achieves a more favorable trade-off between downstream performance and safety retention, demonstrating its effectiveness for reliable LLM fine-tuning.

1 Introduction

Large Language Models (LLMs) are increasingly being deployed in real-world settings where safety is critical [1]. As their capabilities continue to advance, LLMs are assuming increasingly important roles across diverse domains such as healthcare, education, finance, and software engineering [2, 3, 4, 5]. Without adequate safety tuning, LLMs can provide harmful assistance, violate policies, and exhibit unsafe behavior at scale [6]. Ensuring that these models remain helpful while consistently refusing harmful or disallowed requests has therefore become a central challenge in modern LLM development.

In practice, LLM deployment typically involves multiple post-training stages: a base model is first improved through instruction tuning and/or safety tuning, and is then further adapted to downstream tasks, domains, or user-specific objectives [7]. However, such downstream adaptation can degrade previously learned safety behaviors, making it difficult to maintain safety throughout the model life-cycle [8]. This issue becomes more critical as attempts to misuse LLMs have become increasingly sophisticated [9, 10, 11]. Recent work has introduced a wide range of jailbreak strategies, including manually crafted prompts [12], automated prompt optimization [13, 14], adversarial fine-tuning [15, 16], and other adaptive attacks [17, 18, 19, 20] designed to elicit harmful or policy-violating outputs even from safety-tuned models [21]. These threats suggest that safety is an objective that must be preserved throughout model adaptation, deployment, and continual improvement.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

A central challenge in LLM fine-tuning is thus preserving safety during downstream adaptation. To mitigate this issue, a growing body of work has attempted to explicitly preserve safety throughout downstream fine-tuning [22, 23, 24, 25, 26]. The authors of [22] show that adding a small amount of safety data during fine-tuning can provide safety gains, although excessive safety tuning may induce over-refusal. In [23], the authors propose SN-Tune and RSN-Tune, which identify safety-specific neurons and selectively update them to improve safety during downstream adaptation. Other approaches instead modify post-fine-tuning updates: Safe Delta [25] adjusts delta parameters based on their estimated safety degradation and utility contribution, and applies a safety compensation vector. Resta [26] restores safety by adding a safety vector to the compromised model weights. These methods show that safety can be partially preserved or recovered during downstream adaptation.

Challenges. However, despite prior efforts, existing methods share a key limitation: they operate entirely in the *original parameterization* of the model. For example, consider a safety-tuned 2×2 weight matrix W from an arbitrary layer, which can be expressed as

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = w_{11}b_{11} + w_{12}b_{12} + w_{21}b_{21} + w_{22}b_{22}, \tag{1}$$

where $\{w_{ij}\}$ are the model parameters, $\{b_{ij}\}$ are the canonical basis matrices, and each b_{ij} has a single nonzero entry of 1 at position (i, j) and zeros elsewhere (e.g., $b_{11} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$, $b_{22} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$).

In this representation, safety-related information is not localized to a specific coordinate w_{ij}, but is instead distributed across multiple directions in the parameter space (i.e., across w_{11}, w_{12}, w_{21}, w_{22}). Consequently, even if prior methods identify and preserve safety-relevant parameters, downstream adaptation can still overwrite other parameters that encode safety behavior. In other words, preserving a subset of parameters in the original space is often insufficient, as the parameterization does not provide a clear separation between safety-preservation and task adaptation.

This limitation is empirically demonstrated in Table 1. When a safety-tuned model is fine-tuned on a downstream task, full parameter adaptation severely degrades safety. Freezing a subset of parameters (e.g., 10%, 30%, 50%) in the original space during downstream adaptation only partially alleviates this issue. In contrast, applying the same freeze-and-adapt procedure after reparameterization preserves safety much more effectively, while maintaining sufficient capacity for downstream adaptation. These results suggest that robust safety preservation cannot be achieved solely by selecting parameters to freeze in the original space; rather, it requires reparameterizing the model so that safety-preserving and task-adaptive directions are more easily disentangled. We thus pose the following key question:

Table 1: Impact of freezing safety-relevant parameters on safety preservation after downstream fine-tuning. The reported values are averaged over Direct, AutoDAN, PAIR, and PAP attacks using safety-tuned Llama-2-7B.

Fine-tuning setting	**Safety** $\downarrow$
Before FT	52.50
FT (10% frozen)	23.04
FT (30% frozen)	16.01
FT (50% frozen)	12.85
WSR-Tune (10% frozen)	**11.32**

Can we construct a weight-space representation that concentrates safety-relevant information into a compact subspace, while leaving the remaining directions available for downstream adaptation?

Contributions. Motivated by this question, we propose WSR-Tune, a safety-aware LLM fine-tuning approach based on *weight-space reparameterization*. Specifically, we replace the above canonical parameterization in (1) with a reparameterized form as

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = \tilde{w}_{11}\phi_{11} + \tilde{w}_{12}\phi_{12} + \tilde{w}_{21}\phi_{21} + \tilde{w}_{22}\phi_{22}, \tag{2}$$

where ϕ_{ij} indicates a new basis and $\tilde{w}_{ij}$ are the corresponding weights in the reparameterized space. Unlike the canonical basis where each b_{ij} contains a single nonzero entry, the new basis $\{\phi_{ij}\}$ can consist of directions with multiple nonzero entries.

Building on this framework, we construct a safety-conditioned basis $\{\phi_{ij}\}$ and reparameterize the weight space from $\{w_{ij}\}$ to $\{\tilde{w}_{ij}\}$ such that safety-relevant behavior is concentrated in a small subset of directions (i.e., only a few basis directions in $\{\phi_{ij}\}$ capture safety-relevant information), rather than being diffusely distributed. This leaves the complementary directions with sufficient capacity for downstream adaptation, as illustrated in Figure 1. Leveraging the low-rank structure of activations, safety-critical weights can be more clearly identified and isolated in the reparameterized space.

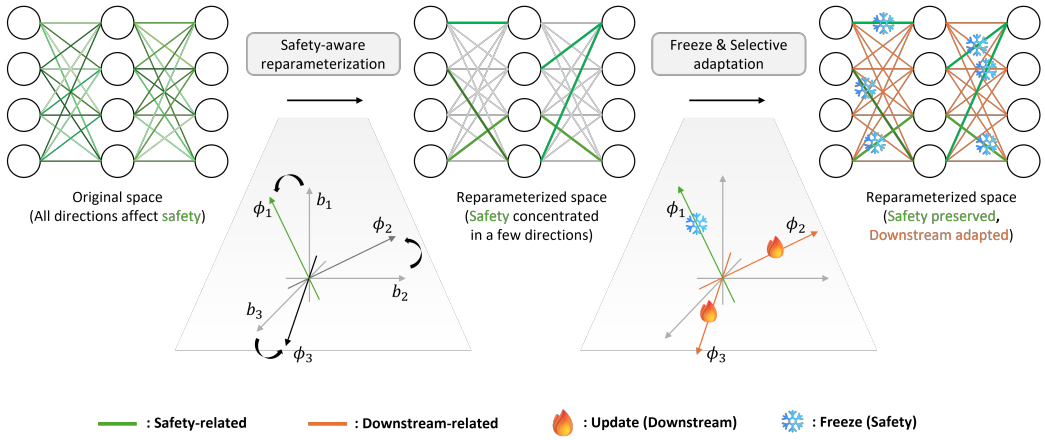

Figure 1: Overview of WSR-Tune. The weight space is reparameterized into a safety-conditioned basis, in which safety-relevant information is concentrated in a subset of coefficients. Safety-critical coefficients are then identified and frozen, while the remaining coefficients are updated for downstream adaptation. This reduces interference between safety and downstream task information during fine-tuning, improving task performance with minimal safety degradation.

We then freeze these weight parameters and update only the complementary ones for downstream adaptation. As a result, our approach reduces interference between previously learned safety behavior and downstream adaptation, enabling more effective safety-preserving fine-tuning.

Our WSR-Tune can be seamlessly integrated with existing downstream adaptation methods originally designed for the original parameter space, by applying them in the reparameterized space without modifying their update rules. Experimental results demonstrate that our method better preserves safety during downstream fine-tuning and achieves a more favorable trade-off between downstream performance and safety retention, compared to methods operating in the original parameter space.

2 Related Works

Preserving safety during downstream adaptation. Recent work on LLM safety has focused on preserving safety during post-training and downstream fine-tuning [15, 27]. SafeInstr [22] shows that utilizing a small amount of safety data during fine-tuning can improve safety, although excessive safety tuning may induce exaggerated safety behaviors. SN-Tune and RSN-Tune [23] take a mechanistic approach by identifying safety-specific neurons and selectively tuning or separating them to improve robustness during downstream adaptation. Other methods instead modify post-fine-tuning updates: Safe LoRA [24] constrains low-rank updates with an alignment-aware projection, Safe Delta [25] adjusts delta parameters in a safety-aware manner while compensating residual safety loss, and RESTA [26] restores safety by adding a safety vector to the fine-tuned model weights. These studies show that safety is highly fragile under downstream adaptation and requires explicit mechanisms for preservation. However, most of these methods still attempt to protect safety within the original parameterization of the model. In contrast, our approach addresses this limitation by reparameterizing the weight space itself, enabling safety preservation through a more structured separation between safety-critical and task-adaptive components.

Analyzing safety in representation and parameter space. Other recent works have analyzed LLM safety through more structured dimensions and spaces [28, 29]. A recent study [30] shows that safety-aligned behavior is not governed by a single refusal direction, but by a multi-dimensional residual space in which a dominant direction captures refusal while additional orthogonal directions encode other safety-related features. A complementary line of work [31] studies safety directly in the original weight space and shows that safety-critical regions can be identified not only at the neuron level but also at the rank level, while remaining vulnerable to downstream fine-tuning attacks even when protected. Safety appears to be highly entangled with general learning rather than isolated in a distinct linear subspace, as both weight-space and activation-space analyses show substantial overlap between safety-related and utility-related directions [32]. Overall, these studies suggest that safety is distributed across directions, ranks, and subspaces rather than being cleanly localized to a small set of canonical parameters [33, 29]. This in turn implies that post-hoc selection of important components in the original space may be insufficient for robust safety preservation.

Subspace-constrained adaptation. Although not focused on safety, a line of prior work studies how to reduce interference by constraining parameter updates within structured subspaces. AlphaEdit [34] projects updates into the null space of preserved knowledge, while orthogonal fine-tuning methods such as PSOFT [35] restrict adaptation to directions that are orthogonal to important components. These methods mitigate interference by carefully selecting update directions, but they still operate within the original parameterization of the model. In a related but distinct direction, [36] explores weight space rotation primarily in the context of class-incremental learning. While this highlights the potential of restructuring the parameter space, it is not designed to address safety preservation during downstream adaptation. These works motivate our view that the safety problem is not only about identifying which parameters matter, but also about choosing a more suitable space in which preservation and adaptation can be disentangled.

3 Algorithm

3.1 Problem setup

We consider adapting an LLM to downstream tasks while preserving its previously learned safety behaviors. Let f_{W_0} denote a safety-tuned model with parameters W_0, where "safety-tuned" indicates that the model has acquired the ability to refuse harmful or disallowed requests through prior safety post-training. In many practical settings, such a model is subsequently adapted to a downstream task using a task-specific dataset D_{task}. However, standard fine-tuning on D_{task} often degrades the model's previously learned safety behaviors, even when the downstream data itself is benign. This leads to an inherent trade-off between improving downstream task performance and preserving safety during adaptation.

Formally, let $\mathcal{L}(W; D)$ denote the loss of model W on dataset D, and let D_{safe} denote a safety-oriented dataset used to preserve safe behavior. In conventional fine-tuning, adaptation is performed by applying a parameter update ΔW to W_0 in the original weight space. Our objective is to improve downstream performance while preserving safety:

$$\min_{\Delta W} \ \mathcal{L}(W_0 + \Delta W; D_{\text{task}}) \ \text{ subject to } \ \mathcal{L}(W_0 + \Delta W; D_{\text{safe}}) \leq \mathcal{L}(W_0; D_{\text{safe}}) + \epsilon, \tag{3}$$

where $\epsilon \geq 0$ denotes a tolerable increase in safety loss.

3.2 Motivation and Insight

Despite extensive efforts, existing approaches [22, 23, 24, 25, 26] operate in the original parameter space. To understand the limitation of this representation, consider an arbitrary $m \times n$ weight matrix

$$W = \begin{bmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{m1} & \cdots & w_{mn} \end{bmatrix} = \sum_{i=1}^{m} \sum_{j=1}^{n} w_{ij} b_{ij} \tag{4}$$

of a *safety-tuned* model, where $\{w_{ij}\}$ are the model parameters and $\{b_{ij}\}$ are the canonical basis matrices. Each b_{ij} has a single nonzero entry of 1 at position (i, j) and zeros elsewhere.

In this coordinate system, safety-related information is not localized to a specific coordinate w_{ij}, but is instead distributed across multiple directions in the parameter space. Consequently, downstream fine-tuning (even when applied to only a subset of parameters) can still overwrite safety-related information encoded in other parameters, leading to degraded safety. This hypothesis is supported by our preliminary results in Table 1, where freezing a certain fraction of parameters in the original space is insufficient to adequately preserve safety. This raises the following question:

Is the original coordinate system the most suitable for fine-tuning, or can we construct a better basis that enables effective downstream adaptation without compromising safety?

3.3 WSR-Tune: Reparameterized Fine-Tuning for Safety Preservation

We propose WSR-Tune, which moves beyond identifying *what* to protect and instead reconsider *how* the parameter space itself is represented. To this end, we transform the weight space into a safety-conditioned coordinate system in which safety-relevant behavior is more compactly concentrated,

making it easier to preserve during downstream fine-tuning. We replace the canonical parameterization

$$W = \sum_{i=1}^{m}\sum_{j=1}^{n} w_{ij} b_{ij} \xrightarrow{\text{Reparameterization}} W = \sum_{i=1}^{m}\sum_{j=1}^{n} \tilde{w}_{ij}\phi_{ij}, \tag{5}$$

where $\{\phi_{ij}\}$ denotes a new basis and $\{\tilde{w}_{ij}\}$ are the corresponding weights in the reparameterized space. Unlike the canonical basis, where each b_{ij} contains a single nonzero entry, the new basis matrices can represent structured directions spanning multiple parameters. The reparameterized weight matrix $\tilde{W}$ can be expressed in terms of the new basis $\{\phi_{ij}\}$ as

$$\tilde{W} = \begin{bmatrix} \tilde{w}_{11} & \cdots & \tilde{w}_{1n} \\ \vdots & \ddots & \vdots \\ \tilde{w}_{m1} & \cdots & \tilde{w}_{mn} \end{bmatrix} = \begin{bmatrix} \langle W, \phi_{11}\rangle & \cdots & \langle W, \phi_{1n}\rangle \\ \vdots & \ddots & \vdots \\ \langle W, \phi_{m1}\rangle & \cdots & \langle W, \phi_{mn}\rangle \end{bmatrix}, \tag{6}$$

where $\langle \cdot, \cdot \rangle$ denotes the Frobenius inner product between two matrices.

This transformation aims to make safety-critical directions more identifiable and concentrated in the reparameterized space, while leaving the complementary directions with sufficient capacity for adaptation. Our key idea is to perform adaptation by updating a subset of $\tilde{w}_{ij}$, rather than directly modifying the original parameters w_{ij}, thereby better preserving safety during downstream adaptation. The high-level idea of WSR-Tune is illustrated in Figure 1. In the following, we describe how our reparameterization is constructed and applied in practice.

Safety-aware weight reparameterization. Our first step is to construct a new basis $\{\phi_{ij}\}$ in which safety-relevant information is concentrated in a small subset of directions. Let $W_l \in \mathbb{R}^{m_l \times n_l}$ denote the weight matrix of the l-th selected layer. For each sample x_i and token position $t \in [T_i]$, let $h_l(x_i, t) \in \mathbb{R}^{n_l}$ denote the input activation of the t-th token to layer W_l, i.e., the hidden representation of that token produced by the preceding layers before being transformed by W_l. We then collect these token-wise activations to form

$$H_l = [\, h_l(x_i, t) \,]_{(i,t)\in\mathcal{T}_{\text{safe}}} \in \mathbb{R}^{n_l \times M}, \tag{7}$$

where T_i denotes the number of valid tokens in sample x_i, and $M = \sum_{i=1}^{N} T_i$ is the total number of token-wise activations collected from samples $\{x_i\}_{i=1}^{N}$ drawn from a safety dataset D_{safe}. We then compute the activation covariance and perform singular value decomposition:

$$H_l H_l^\top = U_l \Sigma_l U_l^\top, \tag{8}$$

where $U_l \in \mathbb{R}^{n_l \times n_l}$ is an orthonormal matrix whose columns are ordered according to the singular values in Σ_l. Figure 2 shows that the singular values exhibit a clear low-rank structure, motivating the use of U_l as an activation-induced basis.

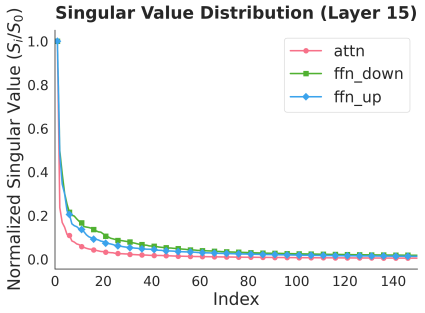

Figure 2: For each layer, singular values are sorted in descending order and normalized by the largest value. Llama-2-7B-Chat model is used.

We thus reparameterize the weight matrix as

$$\widetilde{W}_l = V_l^\top W_l U_l, \tag{9}$$

where U_l is defined above, V_l is an arbitrary unitary matrix, and $\widetilde{W}_l \in \mathbb{R}^{m_l \times n_l}$ denotes the weights in the reparameterized space. Let $\{v_i\}_{i=1}^{m_l}$ and $\{u_j\}_{j=1}^{n_l}$ denote the column vectors of V_l and U_l, respectively. Introducing V_l allows us to define a complete orthonormal basis over the matrix space via outer products of row and column directions, i.e.,

$$\phi_{ij} = v_i u_j^\top, \tag{10}$$

ensuring that each (i, j) corresponds to a distinct basis element. In this work, we set $V_l = I$ for simplicity, which avoids multiple equivalent representations induced by different choices of V_l. Under this formulation, each entry $\widetilde{W}_l(i, j) = \tilde{w}_{ij}$ represents the reparameterized weight along the basis element ϕ_{ij}. This reparameterization defines a coordinate system aligned with activation statistics, in which safety-related information can be represented more compactly.

Identifying safety-critical parameters. Under this reparameterization, downstream adaptation is applied to the coefficients of $\widetilde{W}_l$, rather than to W_l. To preserve safety, within each layer, we estimate the importance of each coefficient using its gradient magnitude on a safety corpus:

$$G_l = \sum_{x \in D_{\text{safe}}} \left| \frac{\partial \mathcal{L}(x)}{\partial \widetilde{W}_l} \right|. \tag{11}$$

Here, $G_l(i,j)$ quantifies the sensitivity of the safety loss to $\widetilde{W}_l(i,j)$, and larger values indicate greater importance for safety retention. Based on G_l, we construct a binary mask $M_l \in \{0,1\}^{m_l \times n_l}$, where $M_l(i,j) = 1$ denotes safety-critical coefficients to be preserved. In practice, we select the top-ρ fraction of entries according to G_l within each layer, where $\rho \in [0,1]$ controls the trade-off between safety retention and downstream adaptability.

Downstream adaptation in the complementary subspace. During downstream fine-tuning, we update only the complementary coefficients while blocking gradients through the masked ones:

$$\widetilde{W}_l \leftarrow \text{stopgrad}(\widetilde{W}_l \odot M_l) + \widetilde{W}_l \odot (1 - M_l), \tag{12}$$

where $\odot$ denotes element-wise multiplication. This constrains adaptation to the complementary subspace, while preserving the coefficients most critical for safety in the reparameterized space.

Our WSR-Tune differs from standard fine-tuning in two key aspects. First, it decouples basis construction from task adaptation: a safety-conditioned basis is constructed once using safety data and kept fixed thereafter. Second, protection is applied in the reparameterized space, where coefficients are aligned with safety-relevant activation directions. This enables more targeted preservation of safety and reduces interference during downstream fine-tuning.

3.4 Discussion

We discuss several practical aspects of WSR-Tune.

Applicability to different layers. WSR-Tune can be applied not only to MLP layers but also to attention layers, including the key, query, and value projection matrices. In our experiments in Section 4, we apply WSR-Tune to both MLP and transformer layers and evaluate its effectiveness.

Compatibility with prior works. WSR-Tune can be integrated with existing methods such as SafeInstr and SN-Tune by applying them in the reparameterized space. We validate this in Section 4.

Freeze ratio. The freeze ratio ρ controls the fraction of coefficients preserved for safety. In Section 4, we study the impact of varying ρ, demonstrating its role in balancing safety and downstream performance. Notably, WSR-Tune achieves a strong trade-off even with relatively small values of ρ.

4 Experiments

4.1 Experimental Setup & Details

Datasets and models. We use the Circuit Breakers dataset [37] as the safety dataset, consisting of harmful prompts paired with safe refusal responses. For downstream adaptation, we use GSM8K [38], Hendrycks MATH [39], MedQA [40], and ARC-Challenge [41]. We conduct experiments on a diverse set of large language models spanning the Llama[42, 43], Qwen [44], and Gemma [45] families, and report the exact model configurations in the corresponding result tables.

Baselines. We compare WSR-Tune with representative safety-preserving fine-tuning baselines: full-parameter fine-tuning, SafeInstr [22], Resta [26], SafeDelta [25], SN-Tune, and RSN-Tune [23]. Full-parameter fine-tuning updates all model weights without safety constraints. SafeInstr is a data-based baseline, for which we add safety examples corresponding to 10% of the downstream training set. Resta and SafeDelta are post-hoc weight-space baselines: Resta adds a safety vector to the fine-tuned model with scaling coefficient 0.3, while SafeDelta adjusts fine-tuning deltas using a safety degradation constraint with $s = 0.1$. SN-Tune and RSN-Tune restrict updates to safety-relevant regions, limiting safety neurons or critical safety neurons to within 1% of the model parameters.

Evaluation metrics. We evaluate downstream performance using 5-shot accuracy on each downstream dataset. For safety evaluation, we use harmful prompts from AdvBench [46] and report attack success rate (ASR). We consider one direct prompting setting using the original AdvBench prompts,

Table 2: Comparison across various chat and instruction-tuned models. Safety is measured by Jailbreak (JB) AVG (averaged over Direct, AutoDAN, PAIR, and PAP; lower is better), and downstream performance is measured by task accuracy (higher is better). We also report changes relative to Full Params FT: Δ_{S} (safety, lower is better) and Δ_{D} (downstream, higher is better). We further define $\Delta_{\mathrm{Overall}} = -\Delta_{\mathrm{S}} + \Delta_{\mathrm{D}}$ to capture the overall trade-off between safety and downstream performance.

	Llama-2-7B-Chat					Llama-2-13B-Chat				
	Safety ↓		Downstream ↑		Overall ↑	Safety ↓		Downstream ↑		Overall ↑
Method	**JB AVG**	Δ_{S}	**GSM8K**	Δ_{D}	$\Delta_{\mathrm{Overall}}$	**JB AVG**	Δ_{S}	**GSM8K**	Δ_{D}	$\Delta_{\mathrm{Overall}}$
Full Params FT	20.78	0.00	41.17	0.00	0.00	10.04	0.00	45.94	0.00	0.00
SafeInstr	13.55	-7.23	38.44	-2.73	+4.50	5.82	-4.22	46.55	+0.61	+4.83
Resta	10.56	-10.22	36.92	-4.25	+5.97	10.34	+0.30	44.20	-1.74	-2.04
SafeDelta	5.57	-15.21	19.79	-21.38	-6.17	0.97	-9.07	28.43	-17.51	-8.45
SN-Tune	27.18	+6.40	38.99	-2.18	-8.58	28.97	+18.93	48.22	+2.28	-16.65
RSN-Tune	20.73	-0.05	40.26	-0.91	-0.86	28.79	+18.75	49.96	+4.02	-14.73
WSR-Tune (Ours)	6.90	-13.88	38.99	-2.18	**+11.70**	1.35	-8.69	49.58	+3.64	**+12.33**

	Llama-3.2-3B-Instruct					Llama-3.1-8B-Instruct				
	Safety ↓		Downstream ↑		Overall ↑	Safety ↓		Downstream ↑		Overall ↑
Method	**JB AVG**	Δ_{S}	**MATH**	Δ_{D}	$\Delta_{\mathrm{Overall}}$	**JB AVG**	Δ_{S}	**MATH**	Δ_{D}	$\Delta_{\mathrm{Overall}}$
Full Params FT	8.65	0.00	21.52	0.00	0.00	9.28	0.00	12.12	0.00	0.00
SafeInstr	8.23	-0.42	22.26	+0.74	+1.16	6.21	-3.07	13.98	+1.86	+4.93
Resta	8.12	-0.53	20.56	-0.96	-0.43	6.74	-2.54	10.56	-1.56	+0.98
SafeDelta	6.48	-2.17	6.48	-15.04	-12.88	4.55	-4.73	1.18	-10.94	-6.21
SN-Tune	21.84	+13.19	19.36	-2.16	-15.35	54.35	+45.07	13.62	+1.50	-43.57
RSN-Tune	24.49	+15.84	19.62	-1.90	-17.74	55.34	+46.06	14.10	+1.98	-44.08
WSR-Tune (Ours)	6.40	-2.25	22.38	+0.86	**+3.11**	5.62	-3.67	13.70	+1.58	**+5.25**

and three jailbreak attack methods, AutoDAN [19], PAIR [17], and PAP [18], which generate attack prompts. We generate jailbreak prompts using the HarmBench [47] and measure ASR with its refusal keywords, counting an attack as successful if the response does not begin with these keywords.

Implementation details. Unless otherwise specified, all methods follow the same downstream fine-tuning pipeline and are trained for 3 epochs, with method-specific learning rates chosen to balance downstream performance and safety retention; full hyperparameters are provided in the Appendix A.1. SafeInstr updates all model weights using the augmented training set, whereas Resta and SafeDelta are applied post hoc to the downstream-tuned checkpoint. For SN-Tune, RSN-Tune, and WSR-Tune, we target all transformer layers and apply the method to the attention projections `q`, `k`, and `v` and the MLP projections `up` and `down`. WSR-Tune constructs the safety-conditioned basis from the Circuit Breakers dataset and uses a freeze ratio of 10% in the reparameterized space. The effect of varying the freeze ratio is further analyzed.

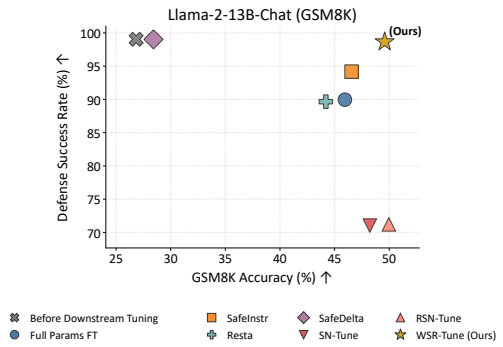

Figure 3: Safety-downstream trade-off on Llama-2-13B-Chat. The x-axis shows downstream accuracy, and the y-axis shows defense success rate (1− ASR), where higher indicates better safety.

4.2 Main Results

Comparison with baselines. We first compare WSR-Tune with representative safety-preserving fine-tuning baselines under downstream adaptation. Table 2 reports the main results across chat and instruction-tuned models. Overall, WSR-Tune achieves the best trade-off between safety and downstream performance across all model settings. WSR-Tune substantially reduces jailbreak attack success while maintaining competitive or improved downstream performance, leading to the largest overall gains over full-parameter fine-tuning. Figure 3 further visualizes this trade-off. Compared with existing baselines, WSR-Tune is positioned closer to the upper-right region, indicating higher downstream accuracy and stronger defense success rate. This further shows that WSR-Tune achieves a favorable balance between safety retention and downstream adaptation. These results indicate that reparameterizing the weight space enables more effective safety preservation than directly constraining updates in the original parameter space.

Table 3: Compatibility of WSR-Tune with existing safety-preserving fine-tuning methods on Llama-2-Chat models. Lower is better for safety metrics, and higher is better for downstream performance. Values are reported as percentages without the percent sign.

Llama-2-7B-Chat	**Safety ↓**					**Downstream ↑**
Method	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**
SN-Tune	0	17.69	45.58	45.46	27.18	38.99
SN-Tune + WSR-Tune	**0**	**11.92**	**41.92**	**38.46**	**23.07**	**41.02**
SafeInstr	0	0.38	18.85	34.96	13.55	38.44
SafeInstr + WSR-Tune	**0**	**0**	**8.46**	**29.69**	**9.53**	**39.27**

Llama-2-13B-Chat	**Safety ↓**					**Downstream ↑**
Method	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**
SN-Tune	**0**	24.42	**33.08**	**58.38**	28.97	48.22
SN-Tune + WSR-Tune	**0**	**7.12**	33.27	**58.38**	**24.69**	**48.75**
SafeInstr	**0**	**0**	9.62	13.65	5.82	46.55
SafeInstr + WSR-Tune	**0**	**0**	**6.54**	**5.19**	**2.93**	**48.60**

Table 4: Additional results on Qwen-2.5-7B-Instruct and Gemma-2-9B-IT. The table follows the same delta-based format as Table 2, reporting changes relative to Full Params FT and the resulting overall safety–downstream trade-off.

	Qwen-2.5-7B-Instruct					**Gemma-2-9B-IT**				
	Safety ↓		**Downstream ↑**		**Overall**	**Safety ↓**		**Downstream ↑**		**Overall**
Method	**JB AVG**	Δ_S	**GSM8K**	Δ_D	$\Delta_{Overall}$	**JB AVG**	Δ_S	**GSM8K**	Δ_D	$\Delta_{Overall}$
Full Params FT	3.62	0.00	67.32	0.00	0.00	5.37	0.00	69.75	0.00	0.00
SafeInstr	6.66	+3.04	67.70	+0.38	-2.66	8.47	+3.10	70.89	+1.14	-1.96
Resta	1.82	-1.80	66.87	-0.45	+1.35	4.58	-0.79	57.85	-11.90	-11.11
SafeDelta	3.82	+0.20	66.87	-0.45	-0.65	4.52	-0.85	64.37	-5.38	-4.53
SN-Tune	37.55	+33.93	69.52	+2.20	-31.73	36.87	+31.50	74.37	+4.62	-26.88
RSN-Tune	38.31	+34.69	71.34	+4.02	-30.67	41.15	+35.78	71.57	+1.82	-33.96
WSR-Tune (Ours)	2.89	-0.73	69.45	+2.13	**+2.86**	4.99	-0.38	70.81	+1.06	**+1.44**

Results on base models. We also evaluate whether the same trend holds when downstream adaptation starts from base models rather than chat or instruction-tuned models. The detailed results are provided in the Appendix C.1, where WSR-Tune consistently achieves the best performance. This suggests that the advantage of weight-space reparameterization is not limited to a specific alignment regime, but also extends to base-to-safe-to-downstream adaptation pipelines.

Compatibility with existing fine-tuning methods. A useful property of WSR-Tune is that it can be combined with existing safety-preserving fine-tuning strategies rather than replacing them entirely. Table 3 shows the performance changes after applying WSR-Tune on top of existing baselines. Across both SN-Tune and SafeInstr, adding WSR-Tune reduces the average attack success rate while also improving downstream performance. These results suggest that the benefits of reparameterization are complementary to existing parameter-selection and safety-data-based strategies, allowing WSR-Tune to serve as a plug-in mechanism for improving the safety-downstream trade-off of prior methods.

4.3 Additional Analyses

Applications to other models. We further evaluate WSR-Tune on Qwen2.5-7B-Instruct and Gemma-2-9B-IT, as shown in Table 4. WSR-Tune consistently preserves safety while maintaining strong downstream performance, suggesting that the benefits of weight-space reparameterization generalize across diverse instruction-tuned model families beyond Llama architectures.

Varying downstream datasets. To assess whether the benefits of WSR-Tune extend beyond mathematical reasoning, we additionally evaluate it on MedQA and ARC-C (Figure 4). These tasks cover diverse settings, including medical question answering and scientific knowledge reasoning. Across all settings, WSR-Tune preserves stronger safety than standard fine-tuning while remaining competitive in downstream performance, indicating that its advantages generalize beyond a single task family.

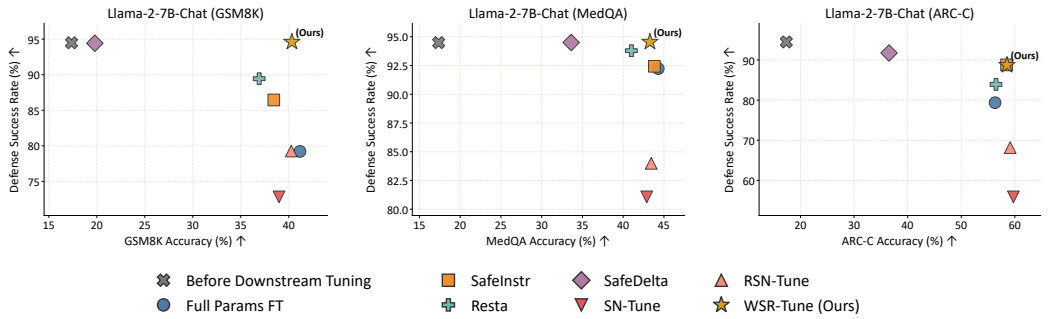

Figure 4: Comparison of safety-preserving downstream fine-tuning methods on Llama-2-7B-Chat under GSM8K, ARC-C and MedQA fine-tuning.

Llama-2-7B-Chat		Safety ↓		Downstream ↑		Overall ↑
Category	**Method**	**JB AVG**	Δ_S	**GSM8K**	Δ_D	**Overall**
Ablation	Full parameter FT	20.78	0.00	41.17	0.00	0.00
	Original space + important mask FT	9.17	-11.61	40.41	-0.76	+10.85
	WSR-Tune	6.90	-13.88	38.99	-2.18	**+11.70**

Table 5: Ablation study on Llama-2-7B-Chat. Lower JB AVG indicates stronger safety, and higher GSM8K indicates better downstream performance.

Ablation study. Table 5 provides an ablation study on Llama-2-7B-Chat, designed to isolate the effect of each component in our method. Full-parameter fine-tuning severely degrades safety, while simply freezing important parameters in the original space improves safety only partially. Applying an importance mask in the original parameter space improves safety by freezing parameters with high safety importance, but the average ASR is still close to 10%. The full WSR-Tune method, which applies safety-aware masking in the reparameterized space, yields the best overall balance, confirming that both components are important for the final performance gain.

Impact of freeze ratio. Table 6 analyzes the effect of the freeze ratio using Llama-2-7B-Chat. Increasing the freeze ratio to 20% freezes more safety-related coefficients, improving safety preservation while reducing the capacity for downstream adaptation. Overall, WSR-Tune remains robust across a moderate range of freeze ratios, maintaining low average ASR and competitive downstream performance.

Table 6: Effect of the freeze ratio in WSR-Tune.

	Safety ↓					Downstream ↑
Freeze ratio	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**
1%	0.00	0.00	10.38	16.88	6.82	38.82
5%	0.00	0.00	9.04	16.42	6.37	39.04
10%	0.00	0.00	9.62	18.00	6.90	38.99
15%	0.00	0.00	9.04	18.00	6.76	39.88
20%	0.00	0.00	8.08	14.27	5.59	38.36

Other experiments. Additional experiments including further analysis across model/task combinations, are deferred to the Appendix C.2.

5 Conclusion and Limitation

In this work, we revisited safety-preserving fine-tuning in LLMs and showed that existing approaches are fundamentally constrained by operating in the original parameter space, where safety information is distributed across directions. We proposed WSR-Tune, a weight-space reparameterization framework that constructs a safety-conditioned basis, enabling downstream adaptation in a complementary subspace. By concentrating safety-relevant information into a small set of coefficients, WSR-Tune allows effective preservation of safety through simple freezing while maintaining strong downstream performance. Our results suggest that safety preservation depends not only on which parameters are updated, but also on how the parameter space is structured, highlighting weight-space reparameterization as a promising direction for reliable LLM adaptation. A limitation of our work is the additional computational overhead introduced by the SVD computation and the estimation of safety-related coefficients; we report the per-epoch training time in the Appendix B. Future work may further combine this perspective with low-rank adaptation, enabling parameter-efficient updates that remain explicitly separated from safety-sensitive directions.

References

[1] Huang et al. Position: TrustLLM: Trustworthiness in large language models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[2] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. BloombergGPT: A large language model for finance, 2023.

[3] Enkelejda Kasneci, Kathrin Sessler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, Jochen Weller, Jochen Kuhn, and Gjergji Kasneci. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 2023.

[4] Mark Chen et al. Evaluating large language models trained on code, 2021.

[5] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[6] Shangbin Feng, Chan Young Park, Yuhan Liu, and Yulia Tsvetkov. From pretraining data to language models to downstream tasks: Tracking the trails of political biases leading to unfair NLP models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11737–11762, 2023.

[7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.

[8] Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. Lisa: Lazy safety alignment for large language models against harmful fine-tuning attack. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[9] Weichen Yu, Kai Hu, Tianyu Pang, Chao Du, Min Lin, and Matt Fredrikson. Infecting LLM agents via generalizable adversarial attack. In *Red Teaming GenAI: What Can We Learn from Adversaries?*, 2025.

[10] Ang Li, Yin Zhou, Vethavikashini Chithrra Raghuram, Tom Goldstein, and Micah Goldblum. Commercial llm agents are already vulnerable to simple yet dangerous attacks, 2025.

[11] Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. Watch out for your agents! investigating backdoor threats to LLM-based agents. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[12] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. Jailbreaking ChatGPT via prompt engineering: An empirical study, 2024.

[13] Yiwei Wang, Muhao Chen, Nanyun Peng, and Kai-Wei Chang. Vulnerability of large language models to output prefix jailbreaks: Impact of positions on safety. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3939–3952, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics.

[14] Yichuan Mo, Yuji Wang, Zeming Wei, and Yisen Wang. Fight back against jailbreaking via prompt adversarial tuning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[15] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024.

[16] Xianjun Yang, Xiao Wang, Qi Zhang, Linda Ruth Petzold, William Yang Wang, Xun Zhao, and Dahua Lin. Shadow alignment: The ease of subverting safely-aligned language models, 2024.

[17] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries, 2024.

[18] Yi Zeng, Hongpeng Lin, Jingwen Zhang, Diyi Yang, Ruoxi Jia, and Weiyan Shi. How Johnny can persuade LLMs to jailbreak them: Rethinking persuasion to challenge AI safety by humanizing LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14322–14350, 2024.

[19] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

[20] James Beetham, Souradip Chakraborty, Mengdi Wang, Furong Huang, Amrit Singh Bedi, and Mubarak Shah. Jailbreaks as inference-time alignment: A framework for understanding safety failures in LLMs. In Vera Demberg, Kentaro Inui, and Lluís Marquez, editors, *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7689–7713, Rabat, Morocco, March 2026. Association for Computational Linguistics.

[21] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[22] Federico Bianchi, Mirac Suzgun, Giuseppe Attanasio, Paul Rottger, Dan Jurafsky, Tatsunori Hashimoto, and James Zou. Safety-tuned LLaMAs: Lessons from improving the safety of large language models that follow instructions. In *The Twelfth International Conference on Learning Representations*, 2024.

[23] Yiran Zhao, Wenxuan Zhang, Yuxi Xie, Anirudh Goyal, Kenji Kawaguchi, and Michael Shieh. Understanding and enhancing safety mechanisms of LLMs via safety-specific neuron. In *The Thirteenth International Conference on Learning Representations*, 2025.

[24] Chia-Yi Hsu, Yu-Lin Tsai, Chih-Hsun Lin, Pin-Yu Chen, Chia-Mu Yu, and Chun-Ying Huang. Safe LoRA: The silver lining of reducing safety risks when finetuning large language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[25] Ning Lu, Shengcai Liu, Jiahao Wu, Weiyu Chen, Zhirui Zhang, Yew-Soon Ong, Qi Wang, and Ke Tang. Safe Delta: Consistently preserving safety when fine-tuning LLMs on diverse datasets. In *Forty-second International Conference on Machine Learning*, 2025.

[26] Rishabh Bhardwaj, Duc Anh Do, and Soujanya Poria. Language models are Homer Simpson! safety re-alignment of fine-tuned language models through task arithmetic. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14138–14149, 2024.

[27] Tiansheng Huang, Gautam Bhattacharya, Pratik Joshi, Joshua Kimball, and Ling Liu. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning attack. In *Forty-second International Conference on Machine Learning*, 2025.

[28] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency, 2025.

[29] Tom Wollschläger, Jannes Elstner, Simon Geisler, Vincent Cohen-Addad, Stephan Günnemann, and Johannes Gasteiger. The geometry of refusal in large language models: Concept cones and representational independence. In *Forty-second International Conference on Machine Learning*, 2025.

[30] Wenbo Pan, Zhichao Liu, Qiguang Chen, Xiangyang Zhou, Yu Haining, and Xiaohua Jia. The hidden dimensions of LLM alignment: A multi-dimensional analysis of orthogonal safety directions. In *Forty-second International Conference on Machine Learning*, 2025.

[31] Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. In *International Conference on Machine Learning*, pages 52588–52610. PMLR, 2024.

[32] Kaustubh Ponkshe, Shaan Shah, Raghav Singhal, and Praneeth Vepakomma. Safety subspaces are not linearly distinct: A fine-tuning case study. In *The Fourteenth International Conference on Learning Representations*, 2026.

[33] Shen Li, Liuyi Yao, Lan Zhang, and Yaliang Li. Safety layers in aligned large language models: The key to LLM security. In *The Thirteenth International Conference on Learning Representations*, 2025.

[34] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Jie Shi, Xiang Wang, Xiangnan He, and Tat-Seng Chua. AlphaEdit: Null-space constrained model editing for language models. In *The Thirteenth International Conference on Learning Representations*, 2025.

[35] Fei Wu, Jia Hu, Geyong Min, and Shiqiang Wang. Efficient orthogonal fine-tuning with principal subspace adaptation. In *The Fourteenth International Conference on Learning Representations*, 2026.

[36] Do-Yeon Kim, Dong-Jun Han, Jun Seo, and Jaekyun Moon. Warping the space: Weight space rotation for class-incremental few-shot learning. In *The Eleventh International Conference on Learning Representations*, 2023.

[37] Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, J Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving alignment and robustness with circuit breakers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[38] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021.

[39] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

[40] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *arXiv preprint arXiv:2009.13081*, 2020.

[41] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv:1803.05457v1*, 2018.

[42] Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models, 2023.

[43] Aaron Grattafiori et al. The Llama 3 herd of models, 2024.

[44] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu,

Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.

[45] Gemma Team et al. Gemma 2: Improving open language models at a practical size, 2024.

[46] Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023.

[47] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Forty-first International Conference on Machine Learning*, 2024.

A Experimental Setup

All experiments are conducted on a server equipped with a single NVIDIA B200 GPU with 180GB of memory, AMD EPYC 9365 36-core processor $\times$ 2 (72 cores total), and 2.2TB of system memory. We use bf16 mixed-precision training with a maximum sequence length of 1024 tokens. For training, we use gradient accumulation to obtain an effective batch size of 16.

A.1 Training Hyperparameters

We describe the training hyperparameters used in our experiments. Unless otherwise specified, all baselines and WSR-Tune are trained for 3 epochs with a safety tuning learning rate of 5×10^{-5} and a downstream fine-tuning learning rate of 5×10^{-5}. This setting is used for all LLaMA-family models in Table 2, the Qwen-2.5-7B-Instruct model in Table 4, and the Llama-2-7B-Chat experiments in Figure 4. The only exception is Gemma-2-9B-IT, for which we use a safety tuning learning rate of 3×10^{-5} and a downstream fine-tuning learning rate of 1×10^{-5}. All experiments use 3 training epochs.

B Runtime and Computational Overhead

We analyze the computational overhead of WSR-Tune compared to standard downstream fine-tuning. All measurements are conducted on the Llama-2-7B-Chat model with GSM8K as the downstream dataset.

Basis construction cost. In WSR-Tune, an additional preprocessing step is required to construct the safety-conditioned basis. This involves performing SVD on all target layers (32 layers, including attention projections `q`, `k`, `v` and MLP projections `up`, `down`). This step takes approximately 12 minutes and requires around 32GB of GPU memory. Notably, this cost is incurred only once before downstream fine-tuning.

Training time comparison. For downstream adaptation, standard full-parameter fine-tuning requires 10 minutes for 3 epochs. In contrast, WSR-Tune requires 20 minutes for the same number of epochs. This corresponds to an additional overhead of approximately $2\times$ in training time.

Summary. Overall, WSR-Tune introduces additional computational overhead due to basis construction and reparameterized training. However, since the basis construction is a one-time cost and the training overhead remains moderate, the method remains practical while providing an improved trade-off between safety and downstream performance.

C Additional Experimental Results

This appendix provides additional experimental results omitted from the main text due to space constraints. In the main paper, Table 2 reports average safety changes across models. Here, we provide the full breakdown for each model, including attack-specific safety results for Direct, AutoDAN, PAIR, and PAP, as well as downstream performance on each evaluated dataset.

C.1 Results on Base Models

Table 7 reports the results when downstream adaptation starts from base models. The upper block presents delta-based metrics relative to Full Params FT, while the lower block provides the corresponding attack-wise raw values. Consistent with the main results, WSR-Tune achieves the best overall trade-off on both base models, preserving safety while maintaining competitive GSM8K performance. These results indicate that the benefit of weight-space reparameterization is not limited to instruction- or chat-tuned models, but also generalizes to base-to-safe-to-downstream adaptation pipelines.

Table 7: Results on base models under GSM8K fine-tuning. The upper table reports delta-based metrics relative to Full Params FT, and the lower table reports the raw attack-wise values. Lower is better for safety metrics, and higher is better for downstream performance.

	Llama-2-7B Base					Llama-3.1-8B Base				
	Safety ↓		Downstream ↑		Overall ↑	Safety ↓		Downstream ↑		Overall ↑
Method	**JB AVG**	Δ_S	**GSM8K**	Δ_D	$\Delta_{Overall}$	**JB AVG**	Δ_S	**GSM8K**	Δ_D	$\Delta_{Overall}$
Full Params FT	18.56	0.00	38.51	0.00	0.00	8.81	0.00	42.38	0.00	0.00
SafeInstr	16.06	-2.50	38.82	+0.31	+2.81	6.56	-2.25	32.83	-9.55	-7.30
Resta	12.56	-6.00	37.38	-1.13	+4.87	8.37	-0.44	38.82	-3.56	-3.12
SafeDelta	11.80	-6.76	35.75	-2.76	+4.00	3.99	-4.82	10.54	-31.84	-27.02
SN-Tune	23.38	+4.82	37.38	-1.13	-5.95	36.85	+28.04	48.90	+6.52	-21.52
RSN-Tune	32.37	+13.81	38.17	-0.34	-14.15	28.54	+19.73	46.02	+3.64	-16.09
WSR-Tune (Ours)	11.32	-7.24	38.82	+0.31	**+7.55**	4.46	-4.35	44.66	+2.28	**+6.63**

C.2 Detailed Results for Main Experiments

Table 8 provides the exact attack-wise results corresponding to Table 2 in the main text. While Table 2 reports the average jailbreak score and delta-based trade-off metrics, Table 8 reports the full values for Direct, AutoDAN, PAIR, and PAP, together with downstream accuracy for each model.

Table 9 provides the exact attack-wise results for the additional model-family experiments summarized in Table 4. Similarly, Table 10 reports the exact values for the downstream dataset experiments visualized in Figure 4. These detailed tables show that the averaged improvements reported in the main text are consistent across individual attack settings and downstream datasets.

Table 8: Comparison of safety-preserving downstream fine-tuning methods across chat and instruction-tuned models. Lower is better for safety metrics, and higher is better for downstream performance.

	Llama-2-7B-Chat						Llama-2-13B-Chat					
	Safety ↓					Downstream ↑	Safety ↓					Downstream ↑
Method	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**
Full Params FT	0	8.27	26.35	48.50	20.78	41.17	0	0.38	15.58	24.19	10.04	45.94
SafeInstr	**0**	0.38	18.85	34.96	13.55	38.44	**0**	**0**	9.62	13.65	5.82	46.55
Resta	**0**	0.38	15.77	26.08	10.56	36.92	0.19	**0**	15.19	25.96	10.34	44.20
SafeDelta	**0**	**0**	**6.92**	**15.35**	**5.57**	19.79	**0**	**0**	**1.54**	2.35	**0.97**	28.43
SN-Tune	**0**	17.69	45.58	45.46	27.18	38.99	**0**	24.42	33.08	58.38	28.97	48.22
RSN-Tune	0.19	5.96	38.85	37.92	20.73	**40.26**	**0**	17.88	37.88	59.38	28.79	**49.96**
WSR-Tune (Ours)	**0**	**0**	9.62	18.00	6.90	38.99	**0**	**0**	3.46	**1.92**	1.35	49.58

	Llama-3.2-3B-Instruct						Llama-3.1-8B-Instruct					
	Safety ↓					Downstream ↑	Safety ↓					Downstream ↑
Method	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**MATH**	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**MATH**
Full Params FT	0.19	0	5.77	28.62	8.65	21.52	0	0	4.04	33.08	9.28	12.12
SafeInstr	0.19	**0**	6.54	26.19	8.23	22.26	**0**	**0**	1.35	23.50	6.21	13.98
Resta	0.19	**0**	7.31	24.96	8.12	20.56	**0**	**0**	2.31	24.65	6.74	10.56
SafeDelta	**0**	**0**	5.19	**20.73**	6.48	6.48	**0**	**0**	**0.38**	**17.81**	**4.55**	1.18
SN-Tune	0.38	**0**	18.46	68.50	21.84	19.36	1.73	74.23	57.50	83.92	54.35	13.62
RSN-Tune	0.96	**0**	26.35	70.65	24.49	19.62	6.54	61.35	68.85	84.62	55.34	**14.10**
WSR-Tune (Ours)	**0**	**0**	**4.23**	21.35	**6.40**	**22.38**	**0**	**0**	1.15	21.31	5.62	13.70

Table 9: Additional results on Qwen-2.5-7B-Instruct and Gemma-2-9B-IT. The upper table reports delta-based metrics relative to Full Params FT, and the lower table reports the corresponding attack-wise raw values. Lower is better for safety metrics, and higher is better for downstream performance.

	Qwen-2.5-7B-Instruct					Gemma-2-9B-IT				
	Safety ↓		Downstream ↑		Overall ↑	Safety ↓		Downstream ↑		Overall ↑
Method	**JB AVG**	Δ_S	**GSM8K**	Δ_D	$\Delta_{Overall}$	**JB AVG**	Δ_S	**GSM8K**	Δ_D	$\Delta_{Overall}$
Full Params FT	3.62	0.00	67.32	0.00	0.00	5.37	0.00	69.75	0.00	0.00
SafeInstr	6.66	+3.04	67.70	+0.38	-2.66	8.47	+3.10	70.89	+1.14	-1.96
Resta	1.82	-1.80	66.87	-0.45	+1.35	4.58	-0.79	57.85	-11.90	-11.11
SafeDelta	3.82	+0.20	66.87	-0.45	-0.65	4.52	-0.85	64.37	-5.38	-4.53
SN-Tune	37.55	+33.93	69.52	+2.20	-31.73	36.87	+31.50	74.37	+4.62	-26.88
RSN-Tune	38.31	+34.69	71.34	+4.02	-30.67	41.15	+35.78	71.57	+1.82	-33.96
WSR-Tune (Ours)	2.89	-0.73	69.45	+2.13	**+2.86**	4.99	-0.38	70.81	+1.06	**+1.44**

	Qwen-2.5-7B-Instruct						Gemma-2-9B-IT					
	Safety ↓					Downstream ↑	Safety ↓					Downstream ↑
Method	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**	**Direct**	**AutoDAN**	**PAIR**	**PAP**	**AVG**	**GSM8K**
Full Params FT	0	0	2.31	12.15	3.62	67.32	0	0.38	0.96	20.15	5.37	69.75
SafeInstr	**0**	**0**	4.81	21.81	6.66	67.70	**0**	2.12	3.46	28.31	8.47	70.89
Resta	**0**	**0**	**0.96**	**6.31**	**1.82**	66.87	**0**	**0**	1.35	16.96	4.58	57.85
SafeDelta	**0**	**0**	1.73	13.54	3.82	66.87	**0**	**0**	0.96	**17.12**	**4.52**	64.37
SN-Tune	1.15	10.96	53.08	85.00	37.55	69.52	**0**	50.38	41.54	55.54	36.87	**74.37**
RSN-Tune	1.35	10.19	58.08	83.62	38.31	**71.34**	0.38	64.23	39.80	60.18	41.15	71.57
WSR-Tune (Ours)	**0**	**0**	1.92	9.65	2.89	69.45	**0**	**0**	**0.77**	19.19	4.99	70.81

Table 10: Comparison of safety-preserving downstream fine-tuning methods on Llama-2-7B-Chat under MedQA and ARC-C fine-tuning. The table follows the delta-based format of Table 2, reporting changes relative to Full Params FT and the resulting overall safety–downstream trade-off.

	Llama-2-7B-Chat (MedQA)					Llama-2-7B-Chat (ARC-C)				
	Safety ↓		Downstream ↑		Overall ↑	Safety ↓		Downstream ↑		Overall ↑
Method	**JB AVG**	Δ_S	**MedQA**	Δ_D	$\Delta_{Overall}$	**JB AVG**	Δ_S	**ARC-C**	Δ_D	$\Delta_{Overall}$
Full Params FT	7.77	0.00	44.30	0.00	0.00	20.64	0.00	56.31	0.00	0.00
SafeInstr	7.57	-0.20	43.83	-0.47	-0.27	11.22	-9.42	58.45	+2.14	+11.56
Resta	6.21	-1.56	41.01	-3.29	-1.73	16.08	-4.56	56.48	+0.17	+4.73
SafeDelta	5.50	-2.27	33.62	-10.68	-8.41	8.26	-12.38	36.52	-19.79	-7.41
SN-Tune	18.93	+11.16	42.89	-1.41	-12.57	44.11	+23.47	59.73	+3.42	-20.05
RSN-Tune	16.01	+8.24	43.44	-0.86	-9.10	31.83	+11.19	59.13	+2.82	-8.37
WSR-Tune (Ours)	5.45	-2.32	43.28	-1.02	**+1.30**	11.29	-9.35	58.53	+2.22	**+11.57**

D Visualization of Baseline Compatibility Results

Figure 5 visualizes the results in Table 3. Each point represents the safety–downstream trade-off of an existing baseline, and the arrow indicates the change after applying WSR-Tune. The figure shows that applying WSR-Tune consistently improves both SN-Tune and SafeInstr on Llama-2-7B-Chat and Llama-2-13B-Chat.

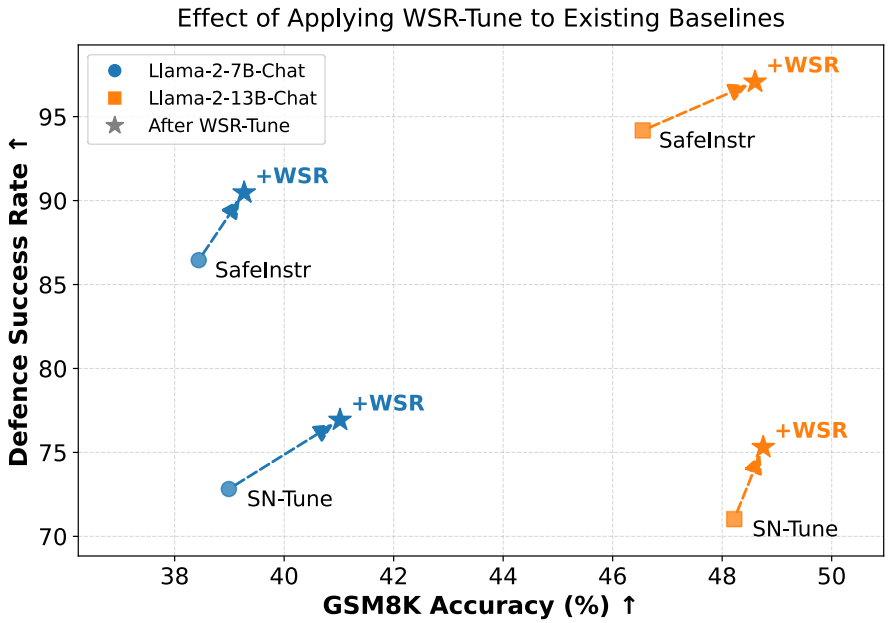

Figure 5: Visualization of the results in Table 3. Each arrow indicates the change from an existing baseline to its WSR-Tune-enhanced variant, where "+WSR" denotes applying WSR-Tune on top of the corresponding baseline. The x-axis denotes GSM8K accuracy, and the y-axis denotes defense success rate. Moving toward the upper-right direction indicates a better safety–downstream trade-off.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly state the main claim that WSR-Tune preserves safety during downstream adaptation by constructing a safety-conditioned reparameterized weight space and freezing safety-critical coefficients. The claims are supported by the method description and experimental comparisons across multiple models and downstream tasks.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper discusses computational overhead as a limitation, including the cost of SVD-based basis construction and safety-critical coefficient estimation. Runtime details are provided in the Appendix.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present formal theoretical results requiring theorem statements or proofs. The mathematical formulations are used to describe the optimization objective, weight-space reparameterization, and masking procedure.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper describes the main experimental setup, including safety and downstream datasets, evaluation metrics, model families, baselines, target modules, and the freeze ratio used for WSR-Tune. The experimental hyperparameters are summarized in the Appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The code is publicly available in a GitHub repository, with instructions for running WSR-Tune and reproducing the main experimental results. The paper also describes the datasets, model backbones, evaluation protocols, and hyperparameter settings needed for reproduction.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies the safety dataset, downstream datasets, evaluation metrics, attack settings, baselines, target modules, and key implementation choices. Training hyperparameters such as epochs and learning rates are described as being provided in the Appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

Justification: The current version reports point estimates for safety and downstream performance but does not yet include error bars, confidence intervals, or statistical significance tests.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper specifies the compute environment, including GPU type (NVIDIA B200), CPU configuration, memory, and training setup in the Appendix. We also report runtime comparisons between standard fine-tuning and WSR-Tune.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research is intended to improve the safety preservation of LLMs during downstream fine-tuning and does not involve deceptive data collection, human-subject experiments, or privacy-sensitive user data. The work uses standard public benchmarks and safety evaluation datasets.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [No]

    Justification: The current version does not include a dedicated broader impacts discussion. However, the work is directly related to LLM safety, and we will include a discussion of potential benefits, such as safer downstream adaptation, and risks, such as limitations or misuse of safety mechanisms, in the final version.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not introduce or release a new high-risk pretrained model, scraped dataset, or deployed system.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cited the original paper that produced datasets used in our work in Section 4.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce any new dataset, model, or benchmark.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve human subjects or human-participant data collection.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The core method does not use an LLM as a non-standard component for generating scientific claims or performing the proposed algorithm. LLMs are the objects of study and evaluation; any use of LLMs for writing or editing assistance does not affect the scientific methodology or results.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.